

Unrealizable

A specification is **unrealizable** when *no controller* can satisfy it. The environment has a strategy to force a violation regardless of controller choices.

Causes:

- Contradictory requirements (e.g., be in two mutually exclusive states).
- Impossible state combinations given the system topology.
- The environment can force the system into a violating state.
- Over-constrained controllable actions.

Diagnostics Block

Enable diagnostics in `.ctxdsl` to get detailed feedback:

```
controllers {
  controller Ctrl {
    source = Plant;
    satisfying = Spec;
    diagnostics {
      counterexample = true;
      deadlock_traces = true;
      max_counter_traces = 100;
      proof_obligations = false;
    }
  }
}
```

| Option             | Purpose                          |
|--------------------|----------------------------------|
| counterexample     | Generate a violating trace       |
| deadlock_traces    | Show traces leading to deadlocks |
| max_counter_traces | Cap the number of traces         |
| proof_obligations  | Show intermediate obligations    |

CLI Flags

Override diagnostics from the command line:

```
mununu context synth file.ctxdsl \
  --formula Spec \
  --automaton Plant \
  --counterexample \
  --deadlock-traces \
  --max-counter-traces 50 \
  --no-proof-obligations \
  --dump-diagnostics diag.json
```

| Flag                   | Effect                       |
|------------------------|------------------------------|
| --counterexample       | Enable counterexample trace  |
| --deadlock-traces      | Enable deadlock trace output |
| --max-counter-traces N | Limit number of traces       |
| --no-proof-obligations | Suppress proof obligations   |
| --dump-diagnostics F   | Write diagnostics to file    |

Diagnostic Output

When synthesis fails, diagnostics report:

1. **Violating initials** — initial states outside the winning region.
2. **Counterexample trace** — a sequence of transitions leading to violation.
3. **Counterstrategy traces** — environment moves that defeat any controller.
4. **Proof obligations** — sub-formulas that failed at each fixpoint iteration.

Debugging Unrealizable Specs

1. **Check violating initials** — which starting states are outside the winning set?
2. **Read the counterexample trace** — follow the sequence of labels to understand the violation path.
3. **Look for contradictions** — are two requirements mutually exclusive?
4. **Compare with a known-realizable formula** — start from a simpler spec that works and add requirements incrementally.
5. **Check controllability** — can the controller actually prevent the bad transitions?
6. **Simplify the environment** — restrict non-determinism to isolate the conflict.

Common Causes

- **Mutually exclusive states** — e.g., requiring `Open && Closed` simultaneously.
- **No guaranteed transitions** — requiring progress but the environment controls all relevant labels.
- **Over-constraining the controller** — leaving no valid moves in certain states.
- **Liveness without fairness** — expecting “always eventually” without environment cooperation.
- **Topology mismatch** — the composition does not connect states the spec assumes are reachable.